ASSIGNMENT 2:

Mansha Bhatnagar

1024150192

2O23

(Q1)

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members(properties):

Name
Roll No
Degree
Hostel
CurrentCGPA

Member Function(behavior):

addDetails();
updateDetails();
updateCGPA();
updateHostel();
displaydetails();

CODE:

E: > ass1.cpp > details

```cpp
#include <iostream>
using namespace std;

struct details{

    string Name;
    string Degree;
    string Hostel;
    int Rollno;
    float CurrentCGPA;

    void adddetails() {
        cout << "Enter the name ";
        getline( cin , Name);
        cout << "Enter the roll number ";
        cin>> Rollno;
        cout << "Enter the Degree " ;
        cin>> Degree;
        cout << "Enter the Hostel ";
        cin>> Hostel;
        cout << "Enter the current CGPA ";
        cin>> CurrentCGPA;
    }

    void updatedetails(){
        cout << "Enter the updated name ";
        cin>> Name;
        cout << "Enter the updated Roll number ";
        cin>> Rollno;
        cout << "Enter the updated Degree ";
        cin>> Degree;
    }

    void updateCGPA(){
        cout << "Enter the updated CGPA ";
        cin>> CurrentCGPA;
    }
```

```cpp
    }

    void updateHostel(){
        cout << "Enter the updated Hostel ";
        cin>> Hostel;
    }

    void displaydetails(){

        cout << "The details are: " << "\n";
        cout << "Name : " << Name << "\n";
        cout << "Roll no. : " << Rollno << "\n";
        cout << "Degree : " << Degree << "\n";
        cout << "Hostel : " << Hostel << "\n";
        cout << "CurrentCGPA : " << CurrentCGPA << "\n";
    }

    };
int main() {
details s;
s.adddetails();
s.updatedetails();
s.updateCGPA();
s.updateHostel();
s.displaydetails();
}
```

OUTPUT:

```
PS E:\> cd "e:\" ; if ($?) { g++ ass1.cpp -o ass1 } ; if ($?) { .\ass1 }
Enter the name Keshav
Enter the roll number 26
Enter the Degree enc
Enter the Hostel M
Enter the current CGPA 10
Enter the updated name Kavya
Enter the updated Roll number 29
Enter the updated Degree coe
Enter the updated CGPA 9.7
Enter the updated Hostel D
The details are:
Name : Kavya
Roll no. : 29
Enter the updated Degree coe
Enter the updated CGPA 9.7
Enter the updated Hostel D
The details are:
Name : Kavya
Roll no. : 29
Degree : coe
Hostel : D
CurrentCGPA : 9.7
```

(Q2):

2. Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public scope.

CODE:

```
#include <iostream>
using namespace std;

 struct details{
    private:
    string Name;
    string Degree;
    string Hostel;
    int Rollno;
    float CurrentCGPA;
    public:
    void adddetails() {
        cout << "Enter the name ";
        getline( cin , Name);
        cout << "Enter the roll number ";
        cin>> Rollno;
        cout << "Enter the Degree " ;
        cin>> Degree;
        cout << "Enter the Hostel ";
        cin>> Hostel;
        cout << "Enter the current CGPA ";
        cin>> CurrentCGPA;
    }

    void updatedetails(){
        cout << "Enter the updated name ";
        cin>> Name;
        cout << "Enter the updated Roll number ";
        cin>> Rollno;
        cout << "Enter the updated Degree ";
        cin>> Degree;
    }
```

```
        cin>> Degree;
    }

    void updateCGPA(){
        cout << "Enter the updated CGPA ";
        cin>> CurrentCGPA;
    }

    void updateHostel(){
        cout << "Enter the updated Hostel ";
        cin>> Hostel;
    }

    void displaydetails(){

        cout << "The details are: " << "\n";
        cout << "Name : " << Name << "\n";
        cout << "Roll no. : " << Rollno << "\n";
        cout << "Degree : " << Degree << "\n";
        cout << "Hostel : " << Hostel << "\n";
        cout << "CurrentCGPA : " << CurrentCGPA << "\n";
    }

    };
int main() {
details s;
s.adddetails();
s.updatedetails();
s.updateCGPA();
s.updateHostel();
s.displaydetails();
}

```

OUTPUT:

```
PS E:\> cd "e:\" ; if ($?) { g++ ass1.cpp -o ass1 } ; if ($?) { .\ass1 }
Enter the name Keshav
Enter the roll number 26
Enter the Degree enc
Enter the Hostel M
Enter the current CGPA 10
Enter the updated name Kavya
Enter the updated Roll number 29
Enter the updated Degree coe
Enter the updated CGPA 9.7
Enter the updated Hostel D
The details are:
Name : Kavya
Roll no. : 29
Enter the updated Degree coe
Enter the updated CGPA 9.7
Enter the updated Hostel D
The details are:
Name : Kavya
Roll no. : 29
Degree : coe
Hostel : D
CurrentCGPA : 9.7
```

(Q3)

Create a code snippet that illustrates the following: Calling of private member functions inside public member function

CODE:

```
#include <iostream> Untitled-1
#include <iostream>
using namespace std;

class Hi {
private:
    void SayHELLO() {
        cout << "Hello there!" << endl;
    }

public:
    void show() {
        cout << "Inside public function" << endl;
        SayHELLO();
    }
};

int main() {
    Hi s;
    s.show();
    return 0;
}
```

OUTPUT:

```
PS C:\Users\HP\Downloads\dsa> cd "c:\U
Inside public function
Hello there!
PS C:\Users\HP\Downloads\dsa>
```

(Q4)

4. Define a class Rectangle with variables width and height of integer typwe along with following:

(a) void getdata() to initialize object values
(b) void calculatearea() to calculate and display the area.

CODE:

Untitled-1.cpp > main()

```cpp
#include <iostream>
using namespace std;
class Rectangle {
private:
    int w, h;
public:
    void getdata() {
        cout << "Enter width: ";
        cin >> w;
        cout << "Enter height: ";
        cin >> h;
    }
    void calculatearea() {
        int area = w * h;
        cout << "Area of Rectangle = " << area << endl;
    }
};
int main() {
    Rectangle r;
    r.getdata();
    r.calculatearea();
    return 0;
}
```

OUTPUT:

```
PS C:\Users\HP\Downloads\dsa> c
Enter width: 40
Enter height: 6
Area of Rectangle = 240
PS C:\Users\HP\Downloads\dsa>
```

(Q5)

5. Define a class Complex with variables real and imaginary along with following:

(a) void setComplex (float, float) to initialize object values.
(b) void displayComplex() to show the complex number
(c) Pass and return objects to calculate sum of two complex numbers. Display the sum.

CODE:

Untitled-1.cpp > ...

```cpp
#include <iostream>
using namespace std;
class Complex {
private:
    float re,im;
public:
    void setComplex(float r, float i) {
        re = r;
        im = i;
    }
    void displayComplex() {
        cout << re << " + " << im << "i" << endl;
    }
    Complex add(Complex c2) {
        Complex temp;
        temp.re = re + c2.re;
        temp.im = im + c2.im;
        return temp;
    }
};
int main() {
    Complex c1, c2, c3;
    c1.setComplex(2.5, 3.1);
    c2.setComplex(4.2, 1.4);
    c3 = c1.add(c2);
    cout << "First Complex Number: ";
    c1.displayComplex();
    cout << "Second Complex Number: ";
    c2.displayComplex();
    cout << "Sum = ";
    c3.displayComplex();
    return 0;
}
```

OUTPUT:

```
First Complex Number: 2.5 + 3.1i
Second Complex Number: 4.2 + 1.4i
Sum = 6.7 + 4.5i
```

(Q6)

6. Implement *scope resolution operator* : : for the following uses:

(a)Class functions defined outside the class
(b) Access a global variable with same name as a local variable
(c)Access a static variables
(d) Use inbuilt libraries (cin cout with scope resolution operator)

CODE:

Untitled-1.cpp > setX(int)

```cpp
#include <iostream>
using namespace std;
// (b) Global variable
int value = 100;
class Demo {
private:
    int x;
public:
    static int count;  // (c) Static variable declaration
    void setX(int x);
    void show();
};
// (c) Define static variable outside the class
int Demo::count = 0;

// (a) Defining member functions outside class
void Demo::setX(int x) {
    this ->x = x;
    count++;
}
void Demo::show() {
    int value = 50; // local variable with same name
    cout << "Local value = " << value << endl;
    cout << "Global value = " << ::value << endl;   // (b) Using :: to access global variable
    cout << "x = " << x << endl;
    cout << "Static count = " << Demo::count << endl; // (c)
}
int main() {
    Demo d1, d2;
    d1.setX(10);
    d2.setX(20);
    d1.show();
    // (d) Using inbuilt libraries with scope resolution
    std::cout << "\nUsing std::cout with scope resolution!" << std::endl;
    return 0;
}
```

OUTPUT:

```
Local value = 50
Global value = 100
x = 10
Static count = 2
```

(Q7):

7. Create a code to implement the *namespace* and use similar variables and functions defined in different code sections.

CODE:

Untitled-1.cpp > ...

```cpp
#include <iostream>
using namespace std;
namespace SectionA {
    int x = 10;
    void show() {
        cout << "SectionA x = " << x << endl;
        cout << "This is SectionA show()" << endl;
    }
}
namespace SectionB {
    int x = 20;
    void show() {
        cout << "SectionB x = " << x << endl;
        cout << "This is SectionB show()" << endl;
    }
}
int main() {
    cout << "Using SectionA::x = " << SectionA::x << endl;
    cout << "Using SectionB::x = " << SectionB::x << endl;
    cout << endl;
    SectionA::show();
    SectionB::show();
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\Downloads\dsa> cd "c:\Users\HP\Downloads\dsa\" ; if ($?)
Using SectionA::x = 10
Using SectionB::x = 20

SectionA x = 10
This is SectionA show()
SectionB x = 20
This is SectionB show()
```